

Padrões de Projeto (Decorator)

QXD0068 - Reuso de Software

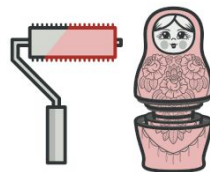
Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Decorator
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação



Decorator

Introdução

- O Decorator (ou Decorador, também conhecido como Wrapper) é um padrão de projeto estrutural que permite adicionar novos comportamentos a objetos dinamicamente, envolvendo-os em “invólucros” (decoradores) que estendem suas funcionalidades sem alterar o código original.

Padrão Decorator

- O Decorator possibilita estender funcionalidades de forma flexível e dinâmica, sem depender de herança.
- Ele mantém a mesma interface do objeto original, permitindo empilhar múltiplos decoradores que adicionam comportamentos de forma incremental.



Problema

- Imagine que você está trabalhando em um biblioteca de notificação que permite que outros programas notifiquem seus usuários sobre eventos importantes.
- A versão inicial da biblioteca foi baseada na classe Notificador que tinha apenas alguns poucos campos, um construtor, e um único método enviar. O método podia aceitar um argumento de mensagem de um cliente e enviar a mensagem para uma lista de emails que eram passadas para o notificador através de seu construtor. Uma aplicação de terceiros que agia como cliente deveria criar e configurar o objeto notificador uma vez, e então usá-lo a cada vez que algo importante acontecesse.
- Em algum momento você se dá conta que os usuários da biblioteca esperam mais que apenas notificações por email. Muitos deles gostariam de receber um SMS acerca de problemas críticos. Outros gostariam de ser notificados no Facebook, e, é claro, os usuários corporativos adorariam receber notificações do Slack.

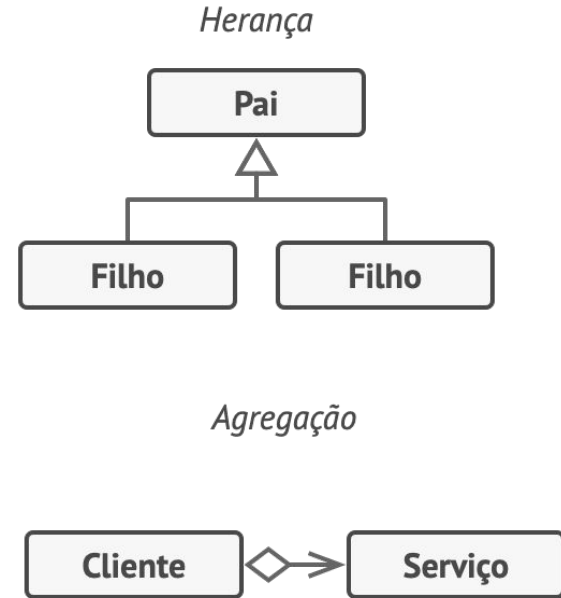
Problema

- Quão difícil isso seria? Você estende a classe Notificador e coloca os métodos de notificação adicionais nas novas subclasses. Agora o cliente deve ser instanciado à classe de notificação que deseja e usar ela para todas as futura notificações.
- Mas então alguém, com razão, pergunta a você, “Por que você não usa diversos tipos de notificação de uma só vez? Se a sua casa pegar fogo, você provavelmente vai querer ser notificado por todos os canais.”
- Você tenta resolver esse problema criando subclasses especiais que combinam diversos tipos de métodos de notificação dentro de uma classe. Contudo, rapidamente você nota que isso irá inflar o código imensamente, e não só da biblioteca, o código cliente também.
- Você precisa encontrar outra maneira de estruturar classes de notificação para que o número delas não quebre um recorde do Guinness acidentalmente.

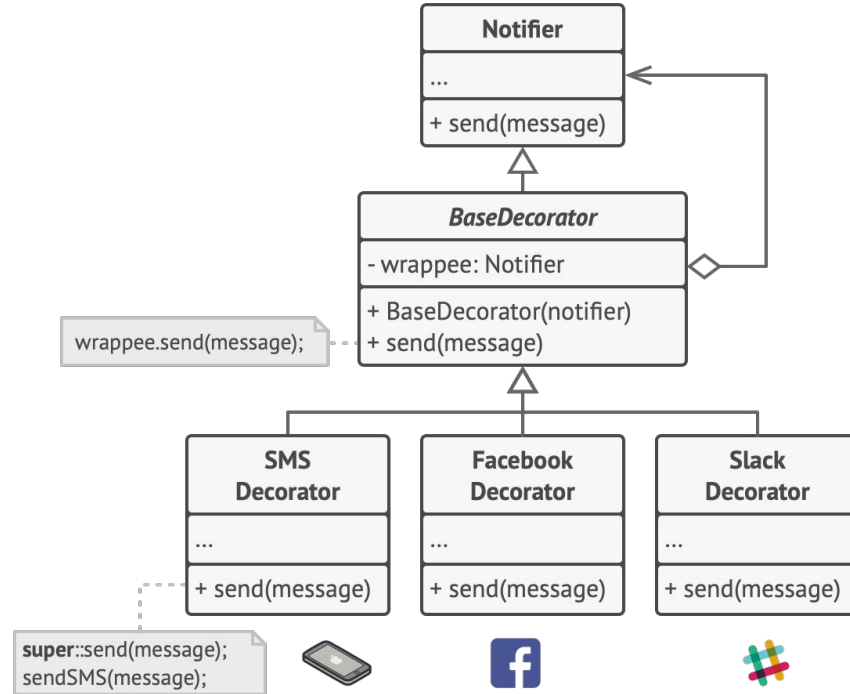
Problema

Solução

- Em vez de usar herança, o padrão propõe usar composição:
 - Cada decorador envolve outro objeto que implementa a mesma interface.
 - O decorador intercepta as chamadas e pode acrescentar comportamento antes ou depois de delegar ao objeto interno.
 - Vários decoradores podem ser empilhados (ex.: e-mail → SMS → Slack).
- Assim, novos comportamentos podem ser combinados em tempo de execução, sem modificar o código existente.



Solução



```

stack = new Notifier()
if (facebookEnabled)
    stack = new FacebookDecorator(stack)
if (slackEnabled)
    stack = new SlackDecorator(stack)

app.setNotifier(stack)
  
```

```

Application
- notifier: Notifier
+ setNotifier(notifier)
+ doSomething()

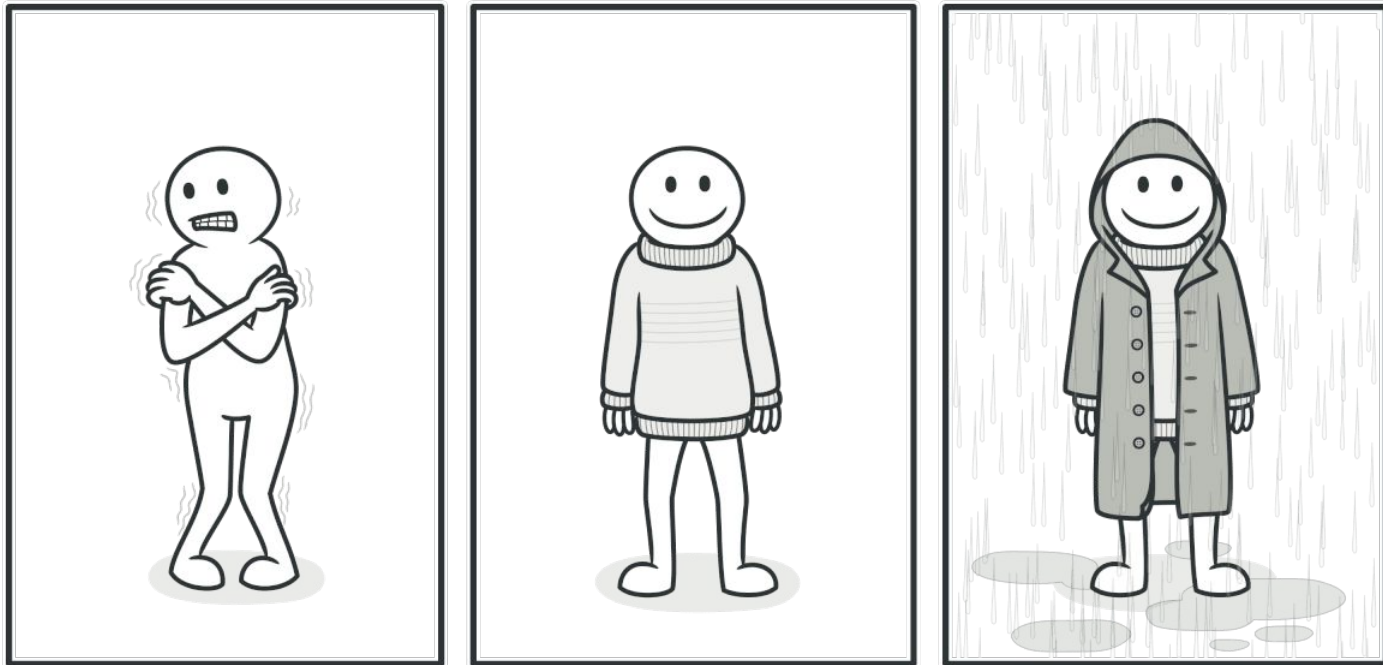
notifier.send("Alerta!")
// Email → Facebook → Slack
  
```



Analogia do Mundo Real

- **Vestir roupas é como usar decoradores:**
 - Você coloca um suéter, depois um casaco, depois uma capa de chuva.
 - Cada peça adiciona uma camada de proteção (comportamento) e pode ser removida facilmente.
 - Você continua sendo a mesma pessoa — apenas com “extensões” externas.

Analogia do Mundo Real



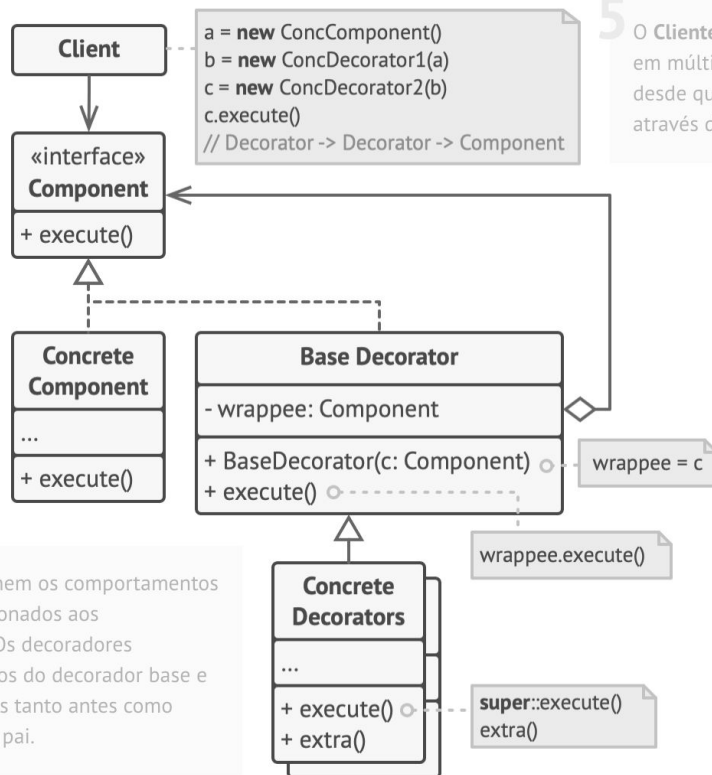
Estrutura do Padrão

- **Componente (interface base):** define as operações comuns.
- **Componente Concreto:** implementação básica do objeto original.
- **Decorator Base:** armazena uma referência para outro componente e delega as chamadas.
- **Decoradores Concretos:** adicionam novos comportamentos (antes/depois da delegação).
- **Cliente:** usa o objeto sem se importar se ele é simples ou decorado.

1 O **Componente** declara a interface comum tanto para os envoltórios como para os objetos envolvidos.

2 O **Componente Concreto** é uma classe de objetos sendo envolvidos. Ela define o comportamento básico, que pode ser alterado por decoradores.

4 Os **Decoradores Concretos** definem os comportamentos adicionais que podem ser adicionados aos componentes dinamicamente. Os decoradores concretos sobrescrevem métodos do decorador base e executam seus comportamentos tanto antes como depois de chamarem o método pai.



5 O **Cliente** pode envolver componentes em múltiplas camadas de decoradores, desde que trabalhe com todos os objetos através da interface do componente.

3 A classe **Decorator Base** tem um campo para referenciar um objeto envolvido. O tipo do campo deve ser declarado assim como a interface do componente para que possa conter ambos os componentes concretos e os decoradores. O decorador base delega todas as operações para o objeto envolvido.

```
// Interface base
interface Notifier {
    fun send(message: String)
}

// Componente concreto
class EmailNotifier : Notifier {
    override fun send(message: String) {
        println("Enviando e-mail: $message")
    }
}

// Decorador base
open class NotifierDecorator(private val wrappee: Notifier) : Notifier {
    override fun send(message: String) {
        wrappee.send(message)
    }
}

// Decoradores concretos
class SMSNotifier(wrappee: Notifier) : NotifierDecorator(wrappee) {
    override fun send(message: String) {
        super.send(message)
        println("Enviando SMS: $message")
    }
}

class SlackNotifier(wrappee: Notifier) : NotifierDecorator(wrappee) {
    override fun send(message: String) {
        super.send(message)
        println("Enviando mensagem no Slack: $message")
    }
}
```

Exemplo em Kotlin

```
// Uso
fun main() {
    val notifier: Notifier = SlackNotifier(SMSNotifier(EmailNotifier()))
    notifier.send("Servidor fora do ar!")
}
```

```
# Interface base
class Notifier:
    def send(self, message):
        pass

# Componente concreto
class EmailNotifier(Notifier):
    def send(self, message):
        print(f"Enviando e-mail: {message}")

# Decorador base
class NotifierDecorator(Notifier):
    def __init__(self, notifier):
        self._notifier = notifier

    def send(self, message):
        self._notifier.send(message)

# Decoradores concretos
class SMSNotifier(NotifierDecorator):
    def send(self, message):
        super().send(message)
        print(f"Enviando SMS: {message}")

class SlackNotifier(NotifierDecorator):
    def send(self, message):
        super().send(message)
        print(f"Enviando mensagem no Slack: {message}")

# Uso
notifier = SlackNotifier(SMSNotifier(EmailNotifier()))
notifier.send("Servidor fora do ar!")
```

Exemplo em Python

Aplicabilidade

- **Use o Decorator quando:**
 - Desejar adicionar comportamentos dinamicamente a objetos, sem alterar seu código.
 - For impossível ou indesejável usar herança (ex.: classes final).
 - Precisar de várias combinações de funcionalidades sem gerar muitas subclasses.

Prós e Contras

- **Vantagens**

- Permite extensão de comportamento sem modificar código existente.
- Comportamentos podem ser combinados em tempo de execução.
- Segue o princípio da responsabilidade única, dividindo funções em classes menores.

- **Desvantagens**

- Pode aumentar a complexidade da estrutura.
- A ordem dos decoradores pode alterar o resultado.
- A pilha de objetos pode ficar difícil de depurar.



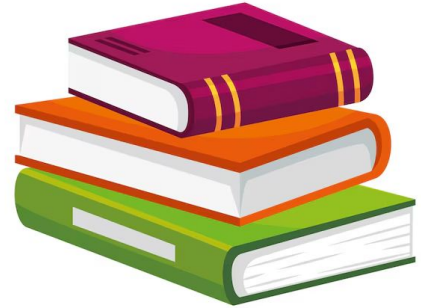
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado! Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

